

Aprendizaje no supervisado: clustering y PCA

Sesión 18 · Descubrir patrones ocultos sin respuestas previas

Tecnológico de Monterrey · TC2001B.601 Ciencia de datos

Prof. Jorge Juvenal Campos Ferreira

Lo que veremos hoy

1

Aprender SIN respuestas: no supervisado

2

Clustering: agrupar lo parecido

3

K-means paso a paso

4

¿Cuántos grupos elegir? (método del codo)

5

Estandarizar antes de agrupar

6

El problema de la dimensión

7

PCA: comprimir información

8

PCA + clustering sobre datos reales

1

Aprender SIN respuestas

¿Qué hacemos cuando no hay etiquetas?

Supervisado vs no supervisado

Definición

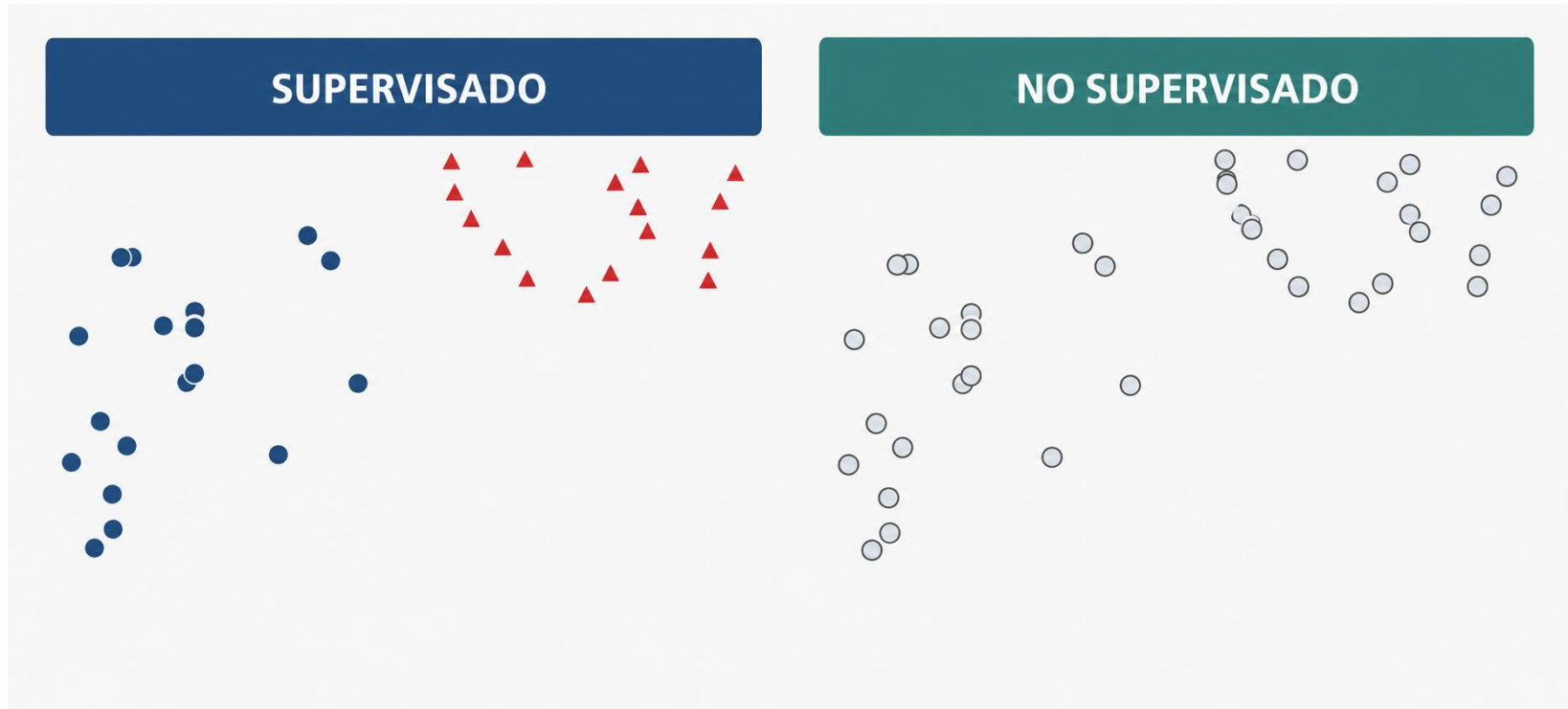
En el aprendizaje SUPERVISADO, cada ejemplo lleva una etiqueta:
"esta solicitud fue aprobada", "este correo es spam".

En el aprendizaje NO SUPERVISADO no hay etiquetas. El algoritmo recibe los datos y debe DESCUBRIR la estructura por sí solo: grupos, patrones, dimensiones relevantes.

En palabras simples

Es como ordenar tu música en playlists sin que nadie te diga las categorías: la computadora escucha las canciones y las agrupa por similitud.

Supervisado vs no supervisado



***Izquierda:** etiquetas conocidas. **Derecha:** el algoritmo las descubre.*

No supervisado = aprender SIN respuestas correctas.
Solo vemos datos y buscamos estructura.

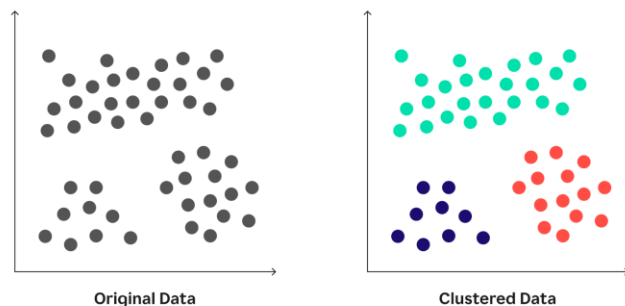
La analogía de la música

Piénsalo así

Imagina que tienes 1,000 canciones sin género.

No sabes si son rock, cumbia o jazz. Pero puedes agruparlas por SIMILITUD: las que tienen bajo ritmo y mucha guitarra van juntas, las de percusión rápida en otro grupo.

Eso es clustering: agrupar SIN saber los nombres de los grupos de antemano.



¿Cuándo usar aprendizaje no supervisado?

Usamos **aprendizaje no supervisado** cuando:

1. No existen etiquetas en los datos.
2. Queremos **EXPLORAR** la estructura antes de definir categorías.
3. La pregunta es '*¿existen grupos naturales?*' (no '*¿cuál es la clase de este punto?*').

Ejemplos: segmentar clientes, agrupar municipios, descubrir perfiles químicos de vinos.

En palabras simples

No hay un examen con respuestas. El objetivo es descubrir, no predecir.

2

PCA:
comprimir información

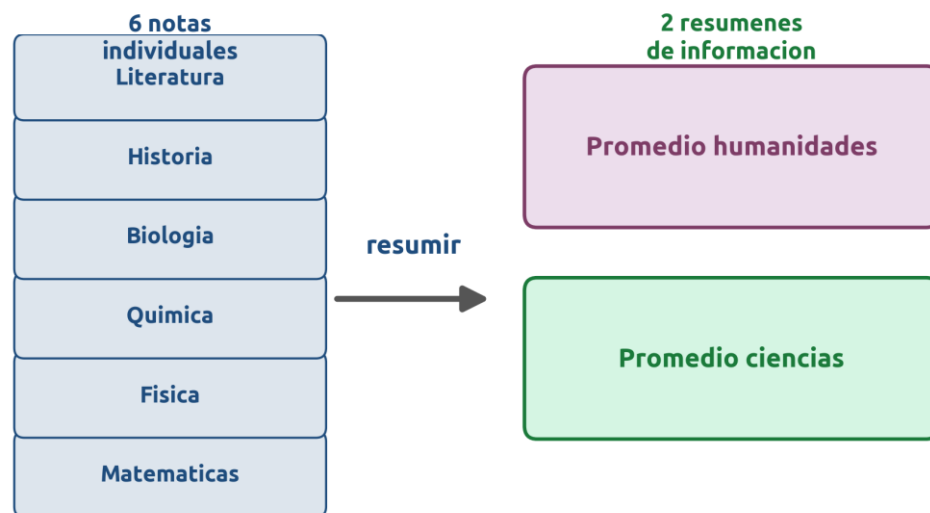
La analogía del promedio de materias

Una analogía:

Si tienes notas en 10 materias, puedes resumirlas en dos promedios: 'ciencias' y 'humanidades'.

Pierdes algo de detalle, pero capturas lo esencial.

PCA hace lo mismo con datos: busca los 'resúmenes' que conservan la MÁXIMA información (varianza). Esos resúmenes son los COMPONENTES PRINCIPALES.



¿Qué es un componente principal?

Definición

Un componente principal (CP) es una **COMBINACIÓN LINEAL** de las variables originales, elegida para capturar la máxima varianza.

CP1 capta la mayor cantidad posible de variación en los datos.

CP2 capta la segunda mayor variación, **PERPENDICULAR** a CP1.

...y así sucesivamente.

Los CP **son NUEVAS variables**; no son las originales.

En palabras simples

CP1 es la dirección en la que los datos 'se esparcen. más'. CP2 es la segunda dirección más importante, sin repetir lo que ya capturó CP1.

¿Qué significan los componentes principales?

Explicación:

Los CP combinan las variables originales con 'pesos' (cargas).

Leer las cargas dice qué variables contribuyen más a cada CP.

Ejemplo: si CP1 tiene cargas altas en '% pobreza', '% carencia educativa' y 'grado de marginación', ese componente representa el 'eje de marginación'.

Interpretar los CP (si queremos hacerlo) requiere conocer el contexto.

En palabras simples

Los CP son combinaciones de tus variables originales.

Leer cuáles variables pesan más en cada CP te dice qué 'concepto' representa ese componente.

El primer componente principal

El **primer componente principal** de un conjunto de variables $X_1, X_2, \dots, X_p$ es la combinación lineal normalizada de las variables

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p$$

que tiene la mayor varianza. Por **normalizada** entendemos que: $\sum_{j=1}^p \phi_{j1}^2 = 1$. *

Llamamos a los elementos $\phi_{11}, \dots, \phi_{p1}$ las cargas (loadings) de la primera componente principal; en conjunto, las cargas forman el vector de cargas de la componente principal:

$$\phi_1 = (\phi_{11} \ \phi_{21} \ \dots \ \phi_{p1})^T$$

*Restringimos las cargas de modo que su suma de cuadrados sea igual a uno, ya que de lo contrario, fijar estos elementos en valores arbitrariamente grandes en valor absoluto podría producir una varianza arbitrariamente grande.

El segundo componente principal

Una vez determinada la **primera componente principal** Z_1 de las variables, podemos encontrar la **segunda componente principal** Z_2 .

La segunda componente principal es la combinación lineal de $X_1, \dots, X_p$ que tiene **varianza máxima** de entre todas las combinaciones lineales que están *no correlacionadas* con Z_1 (son ortogonales). Las puntuaciones de la segunda componente principal $z_{12}, z_{22}, \dots, z_{n2}$ toman la forma

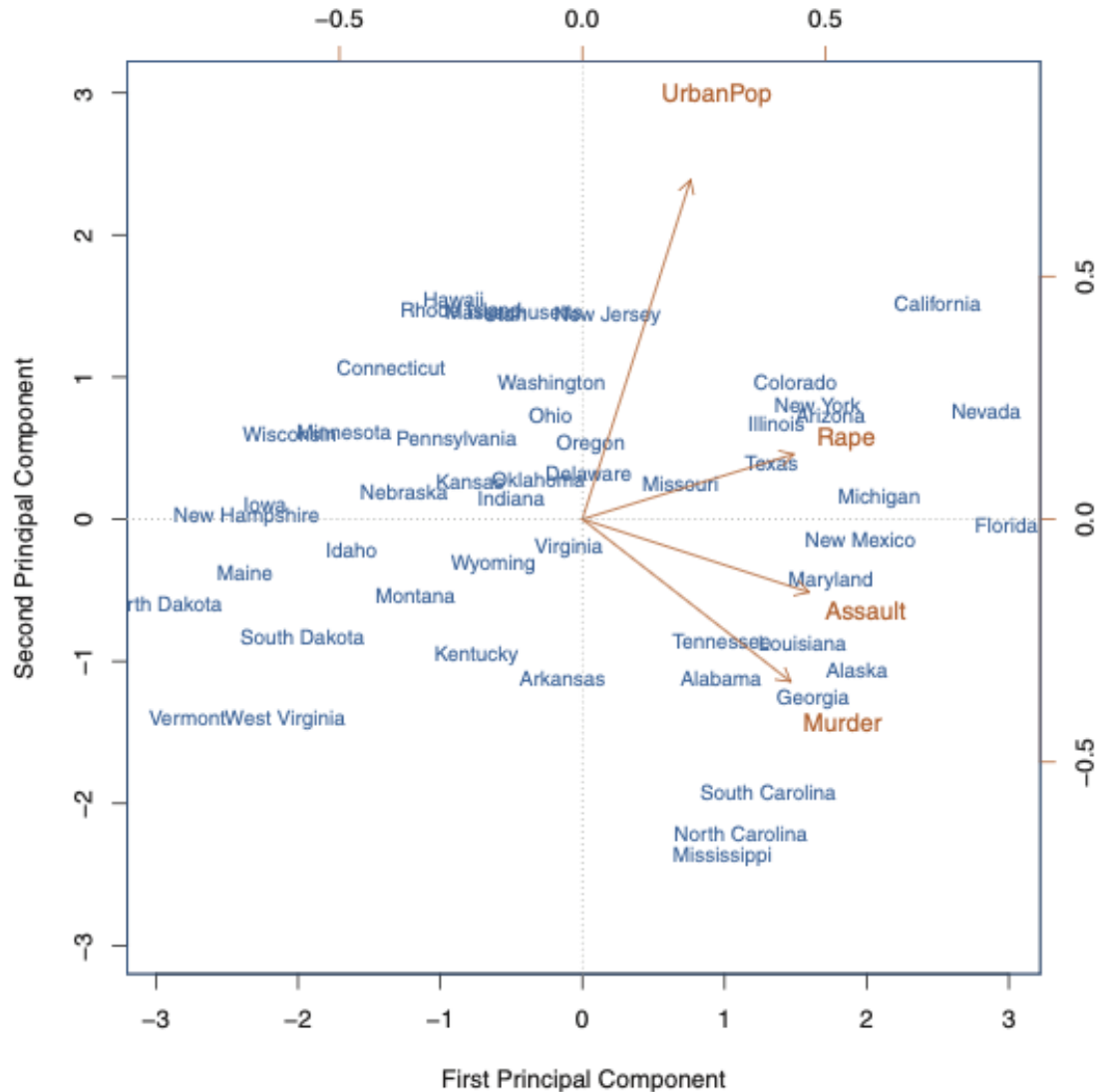
$$z_{i2} = \phi_{12}x_{i1} + \phi_{22}x_{i2} + \dots + \phi_{p2}x_{ip},$$

donde ϕ_2 es el **vector de cargas** de la segunda componente principal, con elementos $\phi_{12}, \phi_{22}, \dots, \phi_{p2}$. Resulta que restringir Z_2 para que esté no correlacionada con Z_1 es equivalente a restringir la dirección ϕ_2 para que sea **ortogonal** (perpendicular) a la dirección ϕ_1 .

Ejemplo de pesos

Componentes principales de la base de arrestos de la base de *USArrests*

	PC1	PC2
Murder	0.5358995	-0.4181809
Assault	0.5831836	-0.1879856
UrbanPop	0.2781909	0.8728062
Rape	0.5434321	0.1673186



PCA: proyectar una nube de puntos a 3D a un plano 2D

Los puntos mantienen su estructura relativa (quién está cerca sigue cerca), pero ahora los vemos en 2 dimensiones.

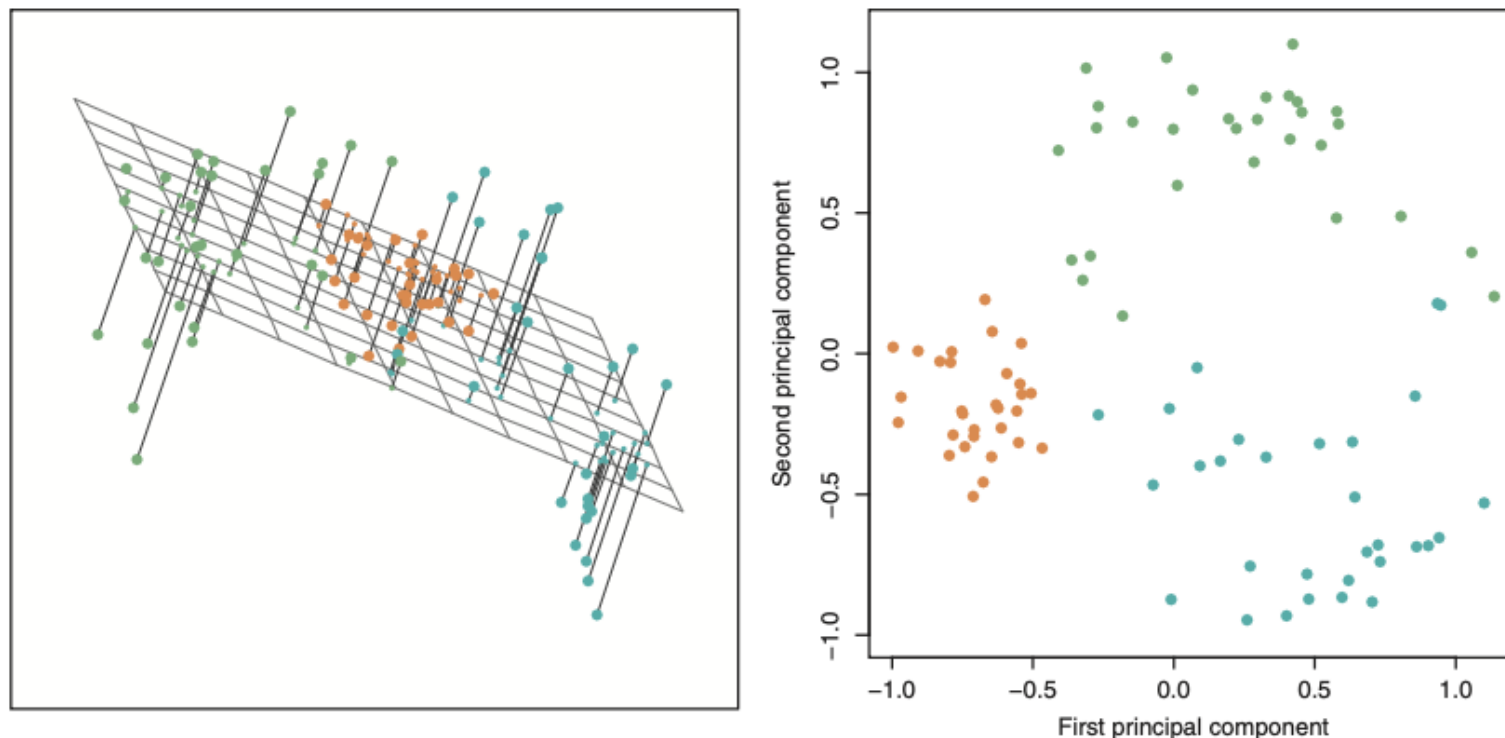
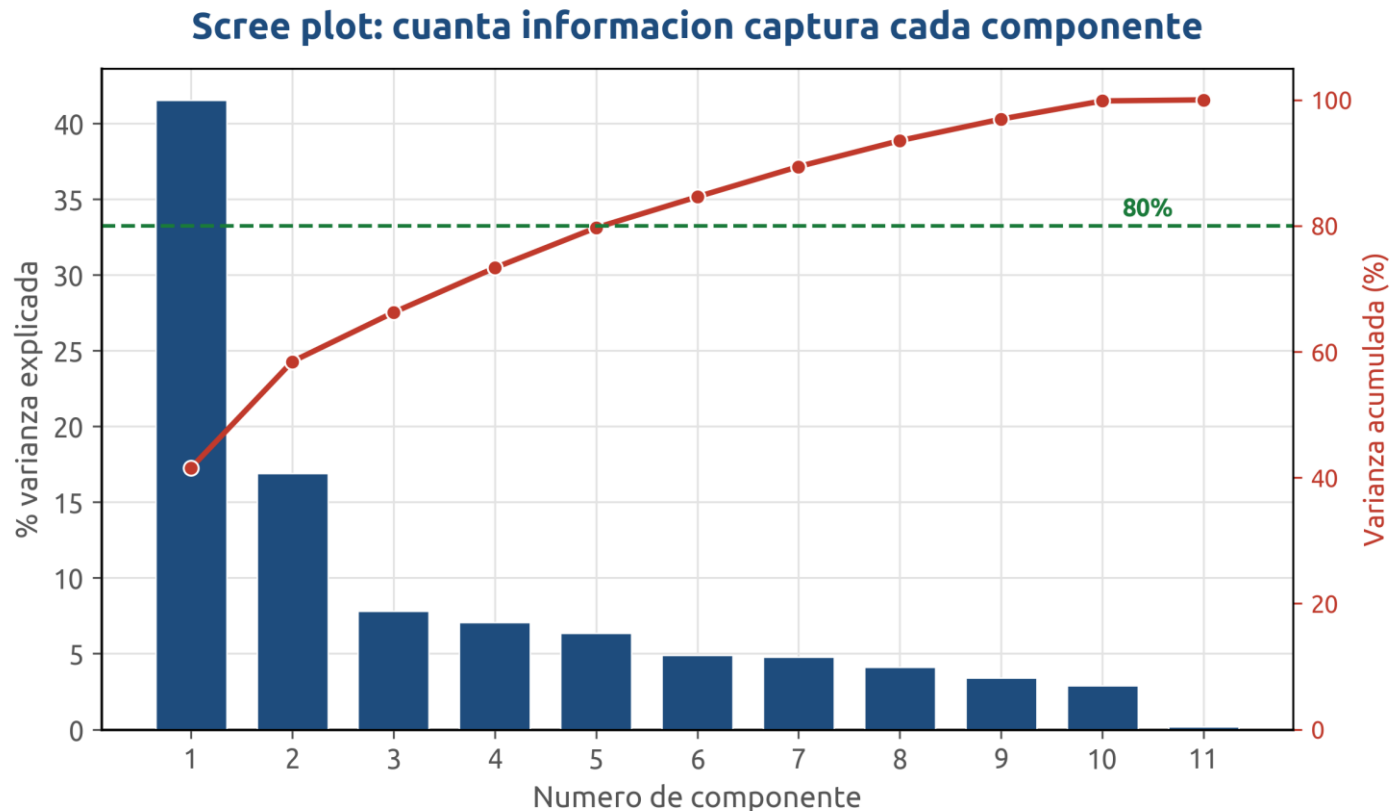


FIGURE 10.2. *Ninety observations simulated in three dimensions. Left: the first two principal component directions span the plane that best fits the data. It minimizes the sum of squared distances from each point to the plane. Right: the first two principal component score vectors give the coordinates of the projection of the 90 observations onto the plane. The variance in the plane is maximized.*

Scree plot: cuánta información captura cada CP



Las barras muestran la varianza de cada componente; la línea roja es la varianza acumulada.

El 'codo' del scree plot indica cuántos componentes conservar. Si los primeros 2-3 suman más del 80%, podemos comprimir bien.

PCA paso a paso (sin fórmulas)

PCA es una receta matemática, pero su lógica es intuitiva.

1

Estandarizar

Z-score de todas las variables. **Indispensable:** PCA es sensible a la escala.

2

Calcular componentes

El algoritmo encuentra las direcciones de máxima varianza en los datos.

3

Elegir cuántos CP conservar

Revisar el scree plot: tomar los CP que sumen el 80-90% de varianza.

4

Proyectar los datos

Transformar cada observación a las nuevas coordenadas (CP1, CP2, ...).

5

Visualizar e interpretar

Graficar en 2D y colorear por grupos; analizar qué representa cada CP.

3

Clustering: agrupar lo parecido

Encontrar grupos naturales en los datos

¿Qué es el clustering?

Definición

Clustering es la tarea de dividir un conjunto de datos en GRUPOS (clusters) de manera que los elementos dentro de un grupo sean SIMILARES entre sí, y diferentes de los elementos en otros grupos.

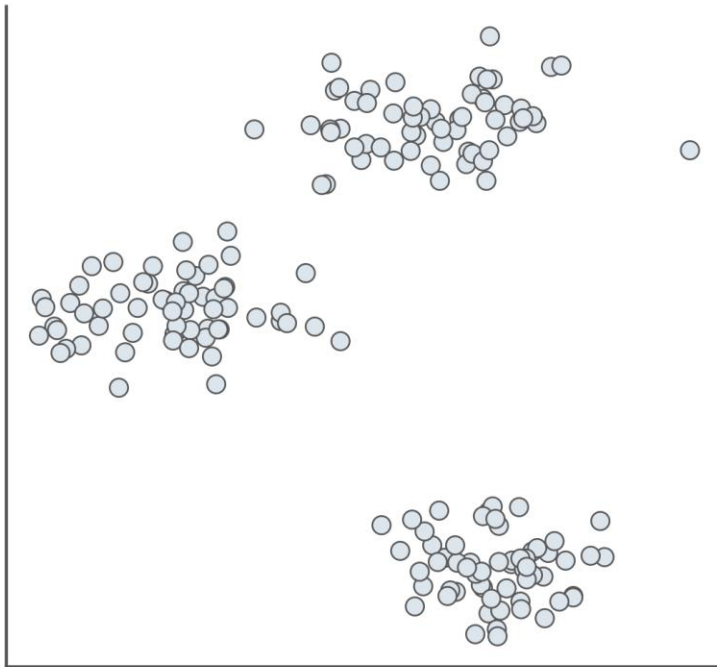
No definimos los grupos de antemano: estos emergen de los mismos datos.

En palabras simples

Como separar frijoles de lentejas (y de piedras) sin saber sus nombres: los pones juntos porque se parecen, no porque alguien te dijo que son del mismo tipo.

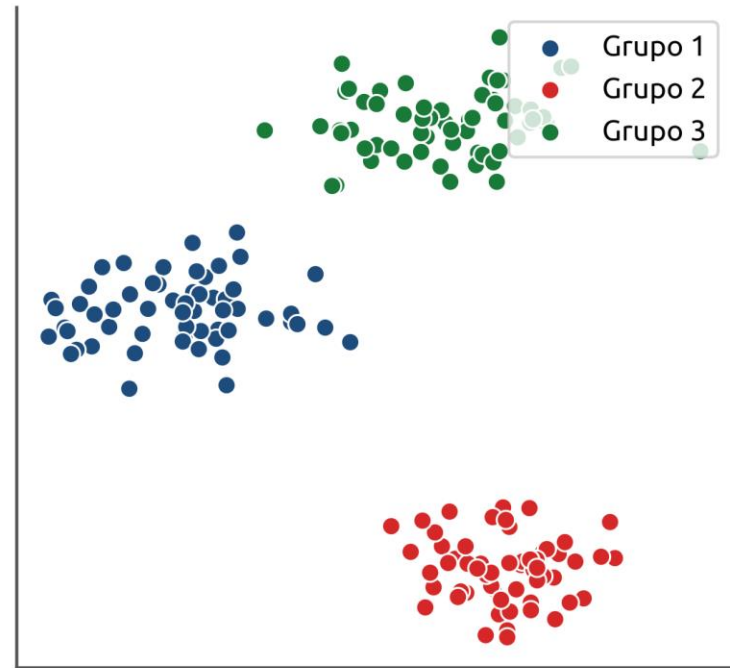
Antes y después del clustering

Antes: puntos sin etiqueta



Solo vemos datos... sin grupos definidos

Después: k-means descubrió 3 grupos



El algoritmo colorio los puntos por similitud

Los mismos puntos: sin etiqueta vs descubriendo 3 grupos.

El algoritmo encontró la estructura que estaba 'escondida' en los datos. Nadie le dijo cuántos grupos había.

Ejemplos reales de clustering

Usos en la vida real

VINOS: 600 muestras con mediciones químicas (acidez, alcohol, azúcar...).

¿Existen perfiles químicos naturales?

MUNICIPIOS: 500 municipios con indicadores sociales (pobreza, internet, ingreso...). ¿Podemos agruparlos para diseñar políticas públicas diferenciadas?

CLIENTES: un banco quiere segmentar a sus clientes sin imponer categorías previas. El clustering las descubre.

4

K-means paso a paso

El algoritmo más popular de clustering

La idea central de k-means

Idea central

K-means divide los datos en K grupos usando CENTROIDES: puntos imaginarios que representan el 'centro' de cada grupo.

La idea: cada punto pertenece al grupo cuyo centroide está MÁS CERCA de él. Y cada centroide se mueve al PROMEDIO de sus puntos asignados. Se repite hasta que nada cambia.

En palabras simples

Es como elegir puntos de reunión: cada persona va al punto más cercano, y el punto se mueve al centro de quien llegó.

La distancia en k-means

Definición

K-means usa la distancia EUCLIDIANA: la misma del Teorema de Pitágoras. Si cada municipio es un punto con 9 coordenadas (una por indicador), la distancia mide qué tan 'parecidos' son dos municipios en todos los indicadores a la vez.

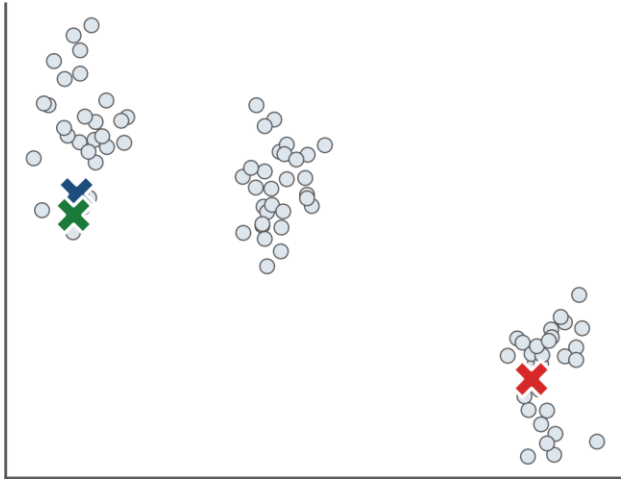
Municipios cercanos en ese espacio tienen perfiles similares.

En palabras simples

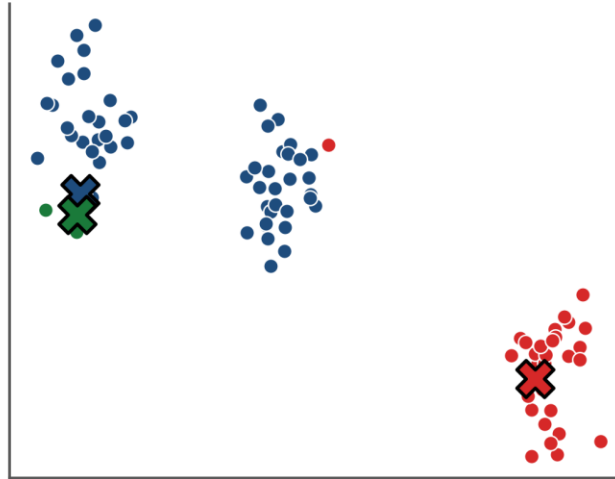
Dos municipios 'cercanos' no (necesariamente) son vecinos geográficos: son municipios con números parecidos en pobreza, internet, ingreso, etc.

K-means: los 4 pasos visualizados

Paso 1: centroides al azar



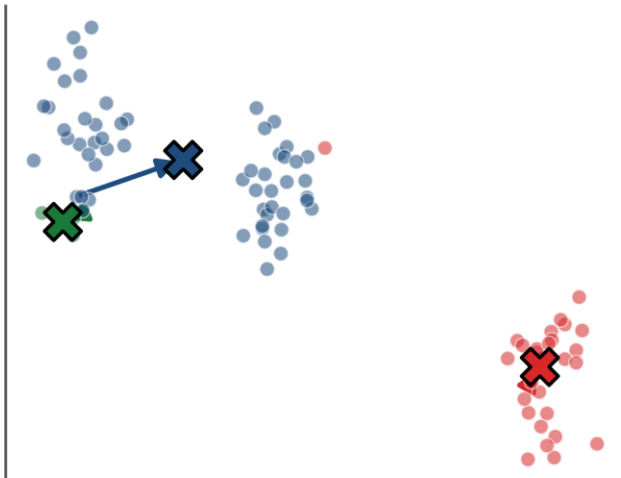
Paso 2: asignar al centroide mas cercano



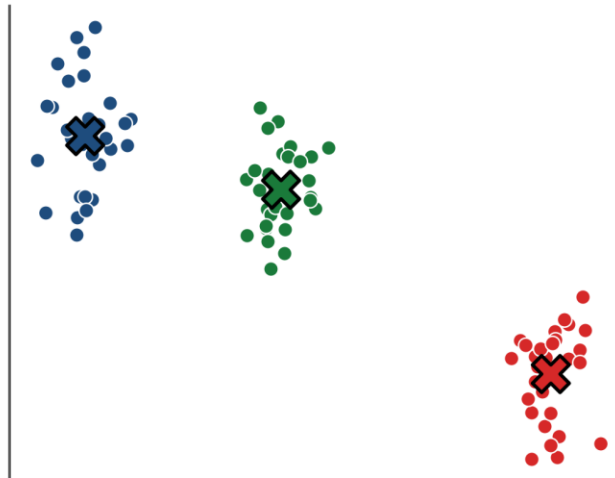
El algoritmo CONVERGE:
eventualmente los centroides dejan de moverse y los grupos quedan fijos.

De centroides al azar a grupos estables en 4 pasos.

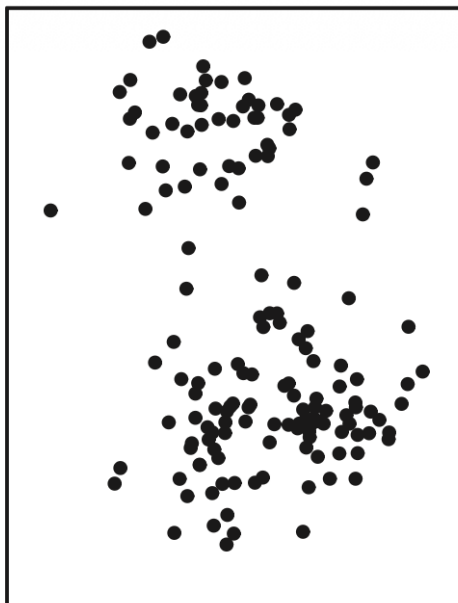
Paso 3: mover al promedio del grupo



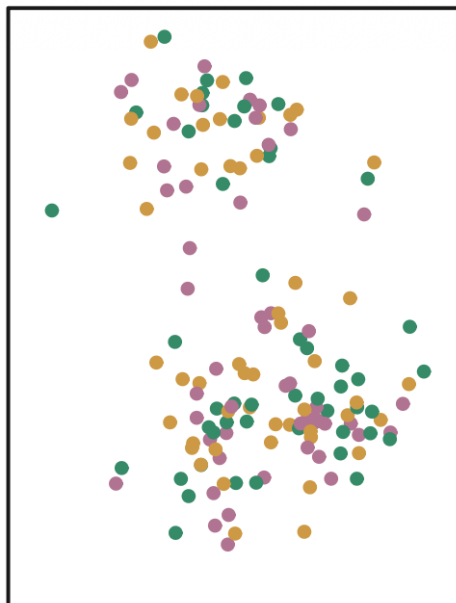
Paso 4: repetir hasta converger



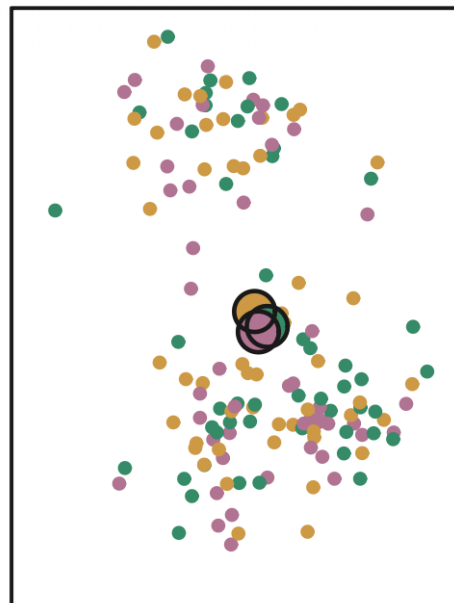
Data



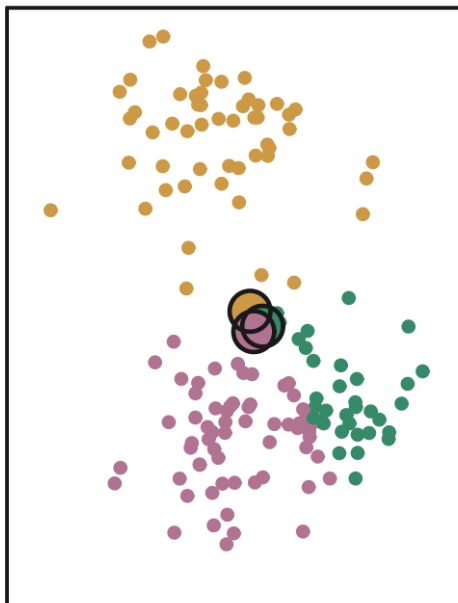
Step 1



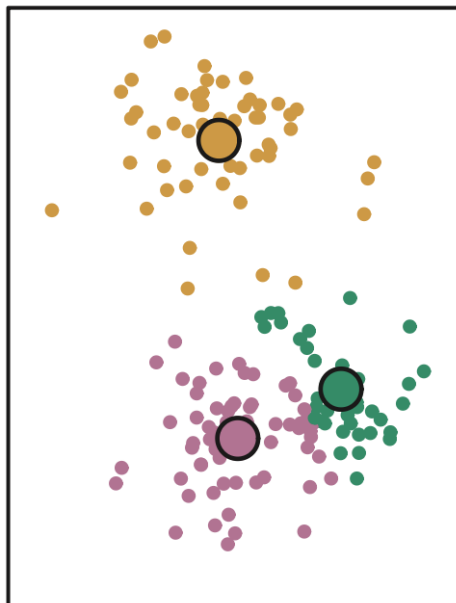
Iteration 1, Step 2a



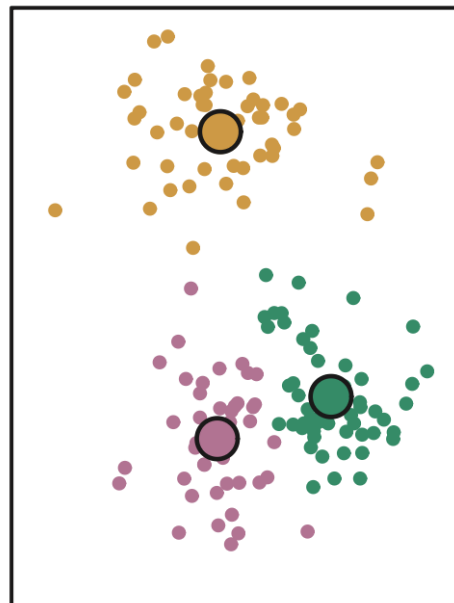
Iteration 1, Step 2b



Iteration 2, Step 2a

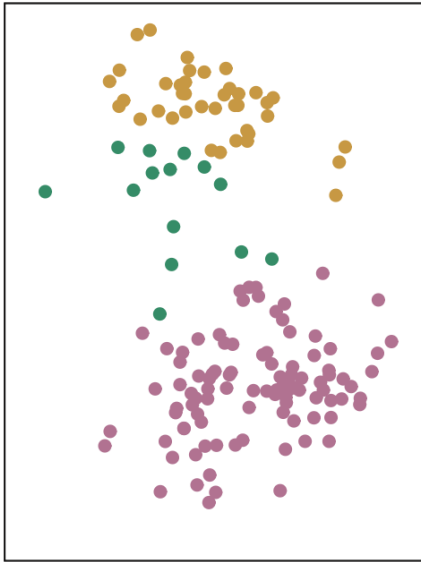


Final Results

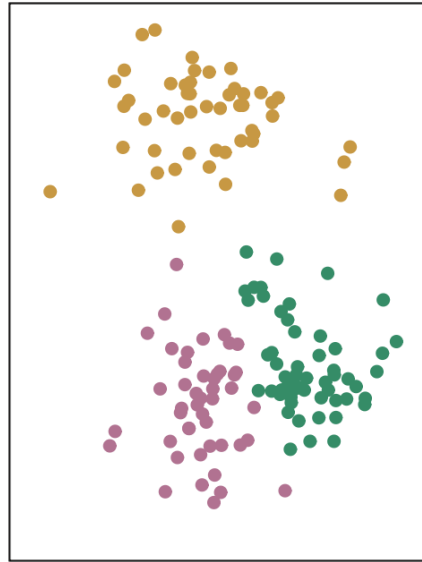


Minimizando la varianza intra-cluster, obtenemos la solución óptima para el reparto en clústers.

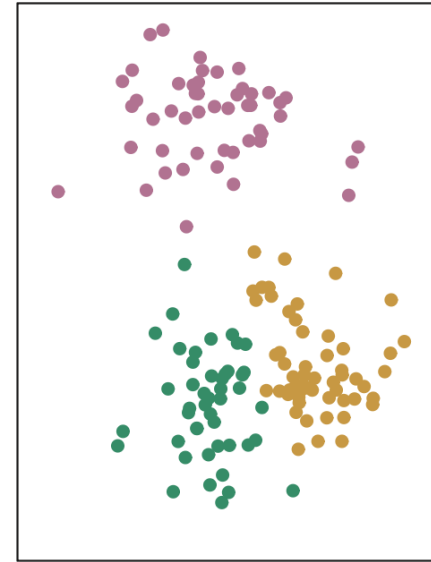
320.9



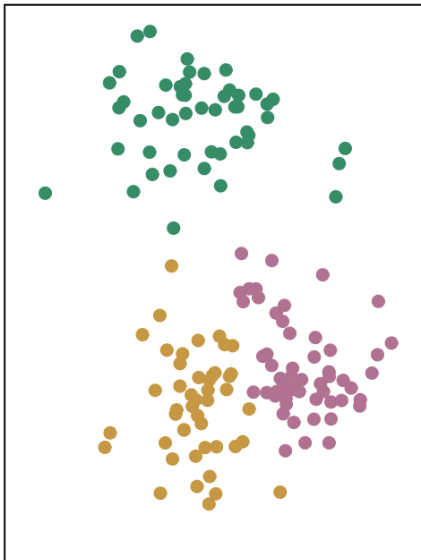
235.8



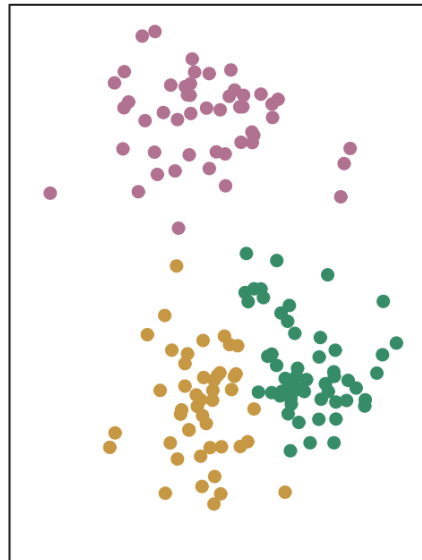
235.8



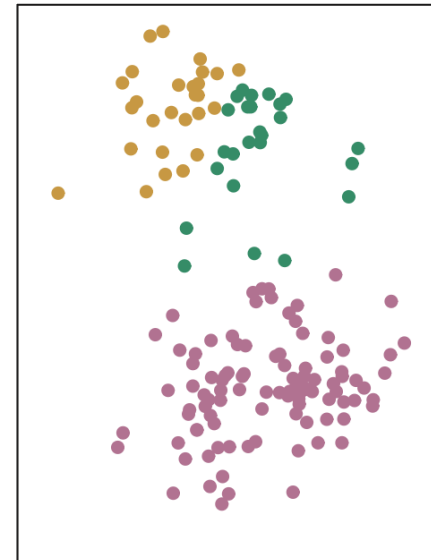
235.8



235.8



310.9



K-means: el algoritmo detallado

Simple, rápido y funciona bien. Es el punto de partida del clustering.

1

Elegir K

Nosotros decidimos cuántos grupos queremos buscar (por ejemplo $K = 3$).

2

Colocar centroides al azar

Elegimos K puntos al azar como centroides iniciales.

3

Asignar cada punto

Calculamos la distancia de cada punto a cada centroide; lo asignamos al más cercano.

4

Mover los centroides

Calculamos el promedio de los puntos de cada grupo; ese promedio es el nuevo centroide.

5

Repetir hasta converger

Repetimos asignar y mover hasta que ningún punto cambia de grupo.

El problema: K es nuestra decisión

El dilema

K-means necesita que NOSOTROS decidamos cuántos grupos hay.

Pero si no conocemos la estructura de los datos de antemano, ¿cómo elegimos K?

No hay una respuesta única '*correcta*'. Pero hay herramientas que nos orientan: el método del codo y el índice de silueta.

En palabras simples

Es como decidir en cuántas categorías ordenar tu cuarto. Hay muchas formas válidas; buscamos la que tenga sentido con los datos.

Inercia: qué tan compactos son los grupos

Definición

La **INERCIA** es la suma de las distancias al cuadrado de cada punto a su centroide. Mide qué tan 'apretados' están los grupos.

Con $K = 1$: inercia máxima (un solo grupo muy disperso).

Con $K = n$: inercia = 0 (cada punto es su propio grupo).

Buscamos el K donde la inercia 'deja de bajar tanto'.

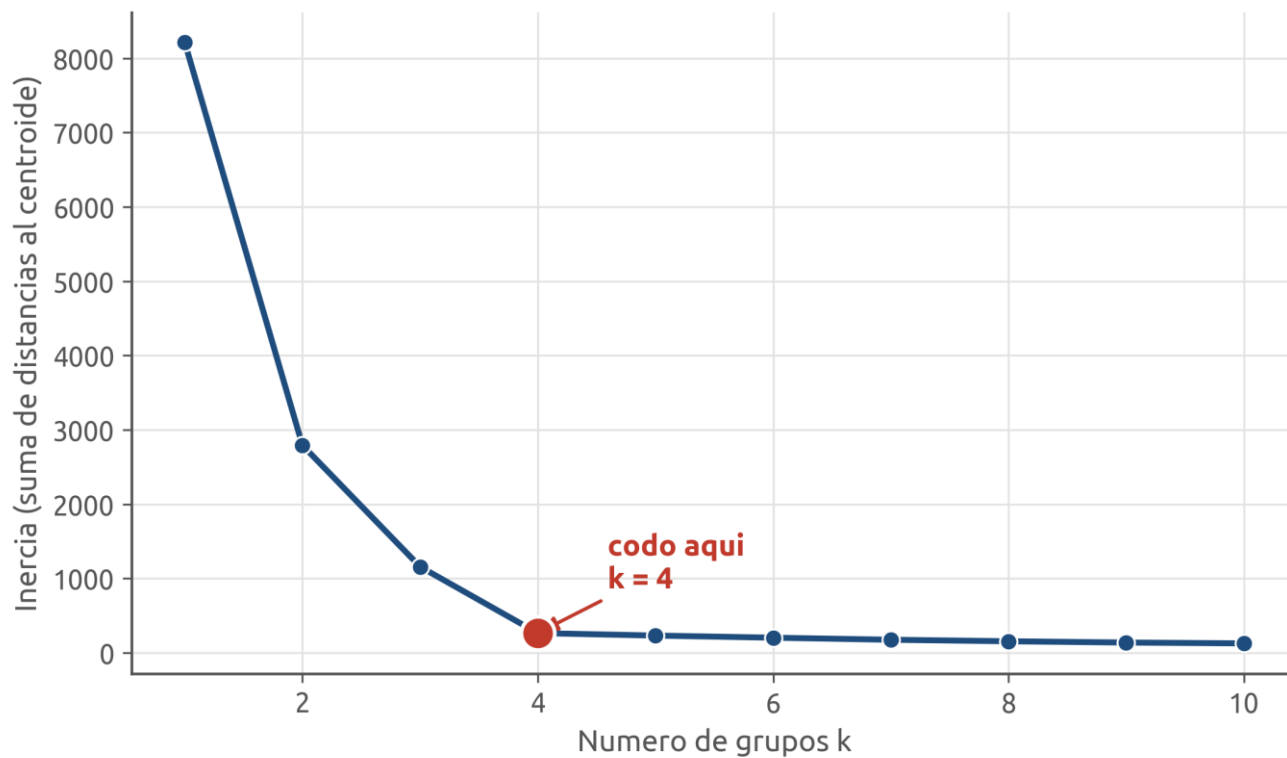
En palabras simples

Si cada persona en la clase es un grupo de uno, la 'distancia al centro' es cero... pero eso no es útil.

Buscamos el número justo de grupos.

Método del codo: la curva de inercia

Metodo del codo: busca donde la curva deja de caer rapido



La inercia baja rápido al principio, luego se estabiliza.

El 'codo' es el K donde agregar más grupos ya no reduce mucho la inercia. Ahí está el número óptimo aproximado.

El codo no siempre es obvio

Cuidado con el codo

En datos reales, la curva del codo a veces NO tiene un quiebre claro. En ese caso:

1. Usar también el índice de silueta (mide separación entre grupos).
2. Probar varios K y evaluar si los grupos tienen INTERPRETACIÓN.
3. Consultar el contexto: un experto en vinos sabe si 3 o 4 perfiles químicos tiene sentido.

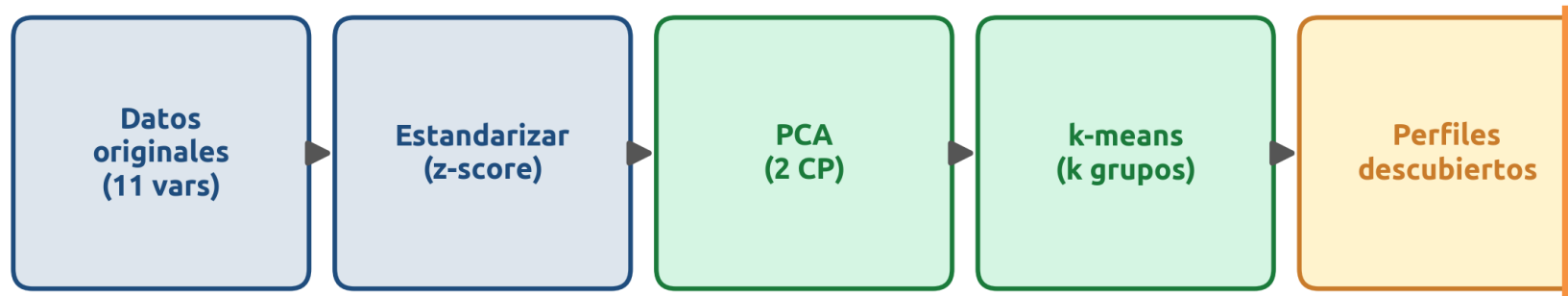
No existe un K 'correcto' universal; depende del problema.

5

Juntando todo: PCA + clustering

Aplicación sobre municipios y vinos

El flujo completo: de datos a perfiles

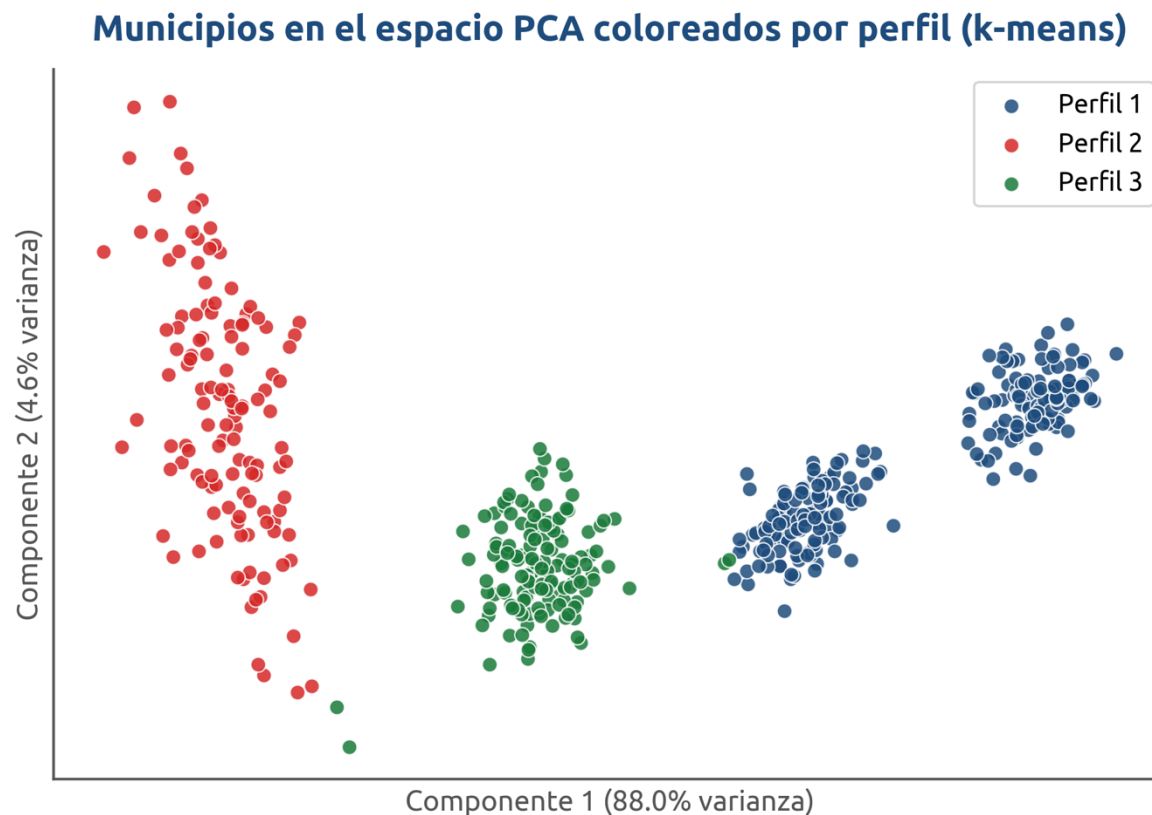


El resultado: grupos con significado, no etiquetas artificiales

Estandarizar -> PCA -> k-means -> interpretar.

PCA facilita la visualización; k-means descubre los grupos. Juntos son una combinación muy poderosa para el aprendizaje no supervisado.

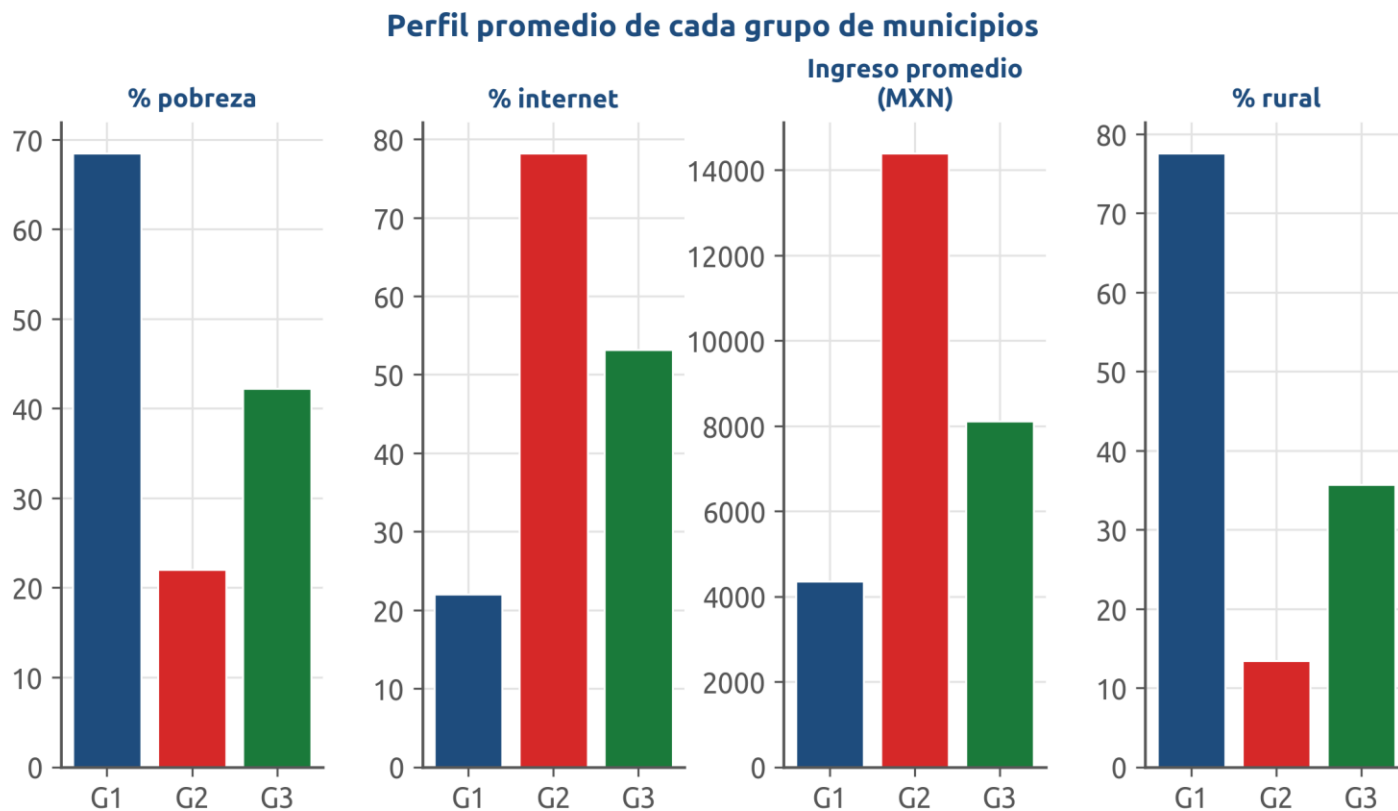
Municipios: 3 perfiles socioeconómicos descubiertos



500 municipios proyectados a 2 componentes, coloreados por k-means.

Los grupos no los definimos nosotros: emergieron de los datos. Ahora podemos interpretar qué tiene en común cada grupo.

¿Qué caracteriza a cada grupo de municipios?

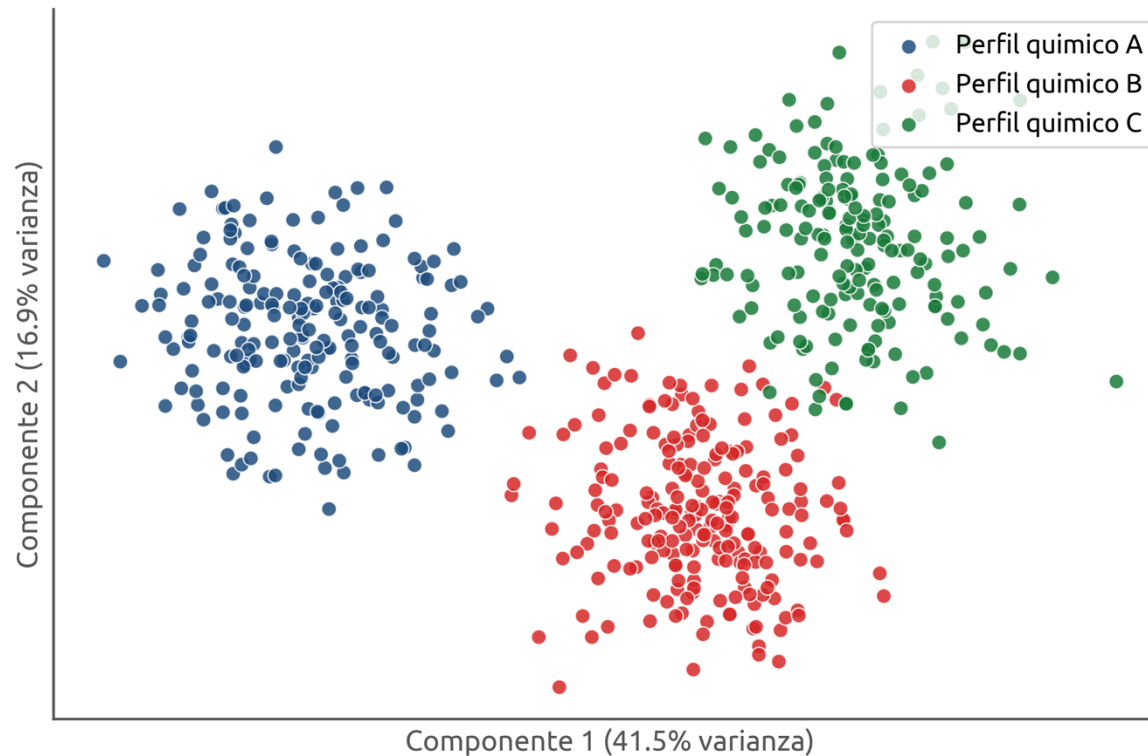


Promedio de 4 indicadores clave por grupo.

Comparar los promedios ayuda a ponerle nombre a los grupos: 'municipios marginados rurales', 'urbanos conectados', etc.

Vinos: 3 perfiles químicos descubiertos

Vinos: 3 perfiles químicos descubiertos con PCA + k-means



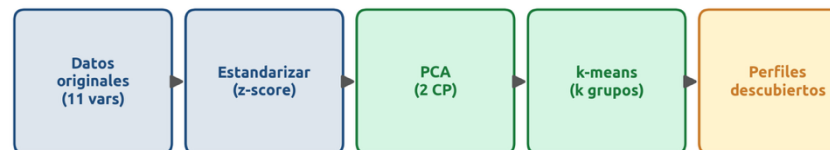
600 muestras de vino en el espacio PCA, coloreadas por k-means.

Sin que nadie etiquetara las muestras, PCA + k-means reveló 3 perfiles químicos distintos.

Pasos para aplicar PCA + clustering en R

Con el paquete tidymodels y factoextra:

1. Cargar datos y quitar columna id.
2. `step_impute_median()` para NAs.
3. `step_normalize()` para z-score.
4. `prcomp()` o `step_pca()` para PCA.
5. `kmeans()` con el K elegido por el codo.
6. `fviz_cluster()` para visualizar.



El resultado: grupos con significado, no etiquetas artificiales

Errores comunes en clustering y PCA

Lista de verificación

1. No estandarizar: una variable domina la distancia.
2. Elegir K mal: usar el codo sin validar interpretación.
3. Interpretar de más: los grupos son exploratorios, no verdad absoluta. Necesitan validación experta.
4. No hay 'accuracy': sin etiquetas no se mide acierto. Evalúa con coherencia e interpretación del dominio.

Conclusiones

Ideas clave para llevarte

- No supervisado = aprender SIN etiquetas; el algoritmo descubre la estructura.
- Clustering agrupa puntos similares; k-means usa centroides y converge iterativamente.
- K es nuestra decisión: el método del codo ayuda, pero requiere interpretación.
- SIEMPRE estandarizar (z-score) antes de clustering o PCA: la escala deforma las distancias.
- PCA comprime muchas variables en componentes que capturan la máxima varianza.
- PCA + k-means: visualizar y agrupar datos de alta dimensión para descubrir perfiles reales.

Funciones a utilizar:

recipe()

Declara una receta SOLO de preprocesamiento (no hay modelo): la fórmula y la secuencia de pasos.

Uso en el ejercicio

```
receta_vinos <- recipe(~ ., data = vinos) %>%  
  step_impute_median(all_numeric_predictors()) %>%  
  step_normalize(all_numeric_predictors())
```

Qué hace

- La fórmula ~ . (sin nada a la izquierda) significa 'no hay variable objetivo': todas las columnas son predictores. Es la marca del no supervisado.
- Encadenamos pasos step_*() con %>%; el ORDEN importa: primero imputar, porque los pasos siguientes no toleran NAs.
- La receta es solo una RECETA: todavía no toca los datos.

step_impute_median()

Rellena los valores faltantes (NAs) de las variables numéricas con su mediana. Red de seguridad.

Uso en el ejercicio

```
step_impute_median(all_numeric_predictors())
```

Qué hace

- Sustituye cada NA por la mediana de esa columna. En vinos no hay NAs, pero incluirlo es buena práctica del flujo.
- La mediana es robusta a valores extremos: en municipios el ingreso es asimétrico (unos pocos muy ricos inflan la media), así que la mediana imputa mejor.
- Importa porque los algoritmos de distancia y el PCA NO toleran NAs.

step_normalize()

Estandariza cada variable a media 0 y desviación 1. Es el paso CRÍTICO antes de PCA y clustering.

Uso en el ejercicio

```
step_normalize(all_numeric_predictors())
```

Qué hace

- El PCA maximiza varianza y el clustering mide distancias: ambos son SENSIBLES A LA ESCALA.
- Sin normalizar, `dioxido_azufre_total` (cientos) aplastaría a `cloruros` o `ácido cítrico` (menos de 1): los grupos reflejarían las unidades, no la química real.
- El z-score deja a todas las variables pesando por igual. Los parámetros (media y sd) se estiman al hacer `prep()`.

prep() + bake()

prep() aprende los parámetros del preprocesamiento; bake() devuelve la matriz ya transformada.

Uso en el ejercicio

```
vinos_norm <- receta_vinos %>%  
  prep() %>%  
  bake(new_data = NULL)  
  
# Verificación: cada columna con media ~0 y desv ~1  
vinos_norm %>%  
  summarise(across(everything(),  
    list(media = mean, desv = sd)))
```

Qué hace

- prep() recorre la receta y aprende los valores que necesita: las medianas de imputación y la media/sd de cada columna.
- bake(new_data = NULL) aplica esos pasos a los mismos datos de la receta y entrega la matriz lista para prcomp() y kmeans().
- Verificamos que media ~0 y desviación ~1 confirma que la normalización quedó bien.

prcomp()

Calcula el PCA: reexpresa las variables en componentes ortogonales, ordenados de mayor a menor varianza.

Uso en el ejercicio

```
pca_vinos <- prcomp(vinos_norm,  
                    center = FALSE, scale. = FALSE)  
  
# Cargas: peso de cada variable en PC1 y PC2  
cargas <- pca_vinos$rotation[, 1:2] %>% round(2)
```

Qué hace

- prcomp() crea ejes (componentes) que son combinaciones lineales de las variables originales; las primeras concentran lo esencial de la variabilidad.
- center y scale. en FALSE porque YA normalizamos con la receta: no hay que escalar dos veces.
- \$rotation guarda las cargas: cuánto pesa cada variable en cada componente, clave para interpretar qué representa PC1 y PC2.

fviz_eig()

Dibuja el scree plot: el porcentaje de varianza explicada por cada componente principal.

Uso en el ejercicio

```
fviz_eig(pca_vinos, addlabels = TRUE,  
         barfill = "#1e4c7d",  
         barcolor = "#1e4c7d",  
         main = "Varianza explicada (scree)")
```

Qué hace

- fviz_eig() (del paquete factoextra) grafica la varianza explicada en orden decreciente.
- Se lee buscando el 'codo': el punto a partir del cual añadir más componentes aporta poco. Ahí decidimos cuántas retener.
- addlabels = TRUE escribe el porcentaje sobre cada barra para leerlo directo.

fviz_pca_biplot()

Proyecta las observaciones en el plano PC1-PC2 y superpone las variables como flechas.

Uso en el ejercicio

```
fviz_pca_biplot(pca_vinos,  
  label = "var", col.var = "#d62828",  
  col.ind = "#1e4c7d",  
  alpha.ind = 0.4)
```

Qué hace

- Los puntos son las 600 muestras de vino; las flechas son las variables originales.
- Variables con flechas casi paralelas están correlacionadas: aquí se ve juntas a las dos de dióxido de azufre (total contiene al libre).
- label = "var" etiqueta solo las variables; col.ind / col.var fijan los colores de puntos y flechas.

fviz_nbclust(method="wss")

Método del codo: grafica la suma de cuadrados dentro de los grupos (WSS) para cada k.

Uso en el ejercicio

```
set.seed(123)
fviz_nbclust(vinos_norm, kmeans,
             method = "wss", nstart = 20) +
  labs(title = "Método del codo (WSS)")
```

Qué hace

- Al aumentar k, la WSS siempre baja. Buscamos el 'codo': el k a partir del cual el descenso se vuelve marginal.
- Recibe la matriz y la función de clustering (kmeans); nstart = 20 hace 20 arranques por cada k para estabilizar la curva.
- set.seed() hace reproducible la aleatoriedad de los arranques.

fviz_nbclust(method="silhouette")

Método de la silueta: mide qué tan bien separados quedan los grupos para cada k.

Uso en el ejercicio

```
set.seed(123)
fviz_nbclust(vinos_norm, kmeans,
  method = "silhouette", nstart = 20) +
  labs(title = "Método de la silueta")
```

Qué hace

- La silueta evalúa, para cada punto, qué tan cómodo está en su grupo vs. qué tan cerca queda del grupo vecino.
- El k con silueta promedio MÁS ALTA es el preferido; suele desempatar cuando el codo es ambiguo.
- Tras leer ambos diagnósticos, en vinos elegimos k = 3.

kmeans()

Agrupar las observaciones en k grupos minimizando la distancia de cada punto al centroide de su grupo.

Uso en el ejercicio

```
set.seed(123)
k_elegido <- 3
km_vinos <- kmeans(vinos_norm,
  centers = k_elegido,
  nstart = 25)
km_vinos$size # tamaño de cada grupo
```

Qué hace

- centers = k fija de antemano el número de grupos (decidido con codo y silueta).
- nstart = 25: k-means depende de la inicialización aleatoria de los centroides; probar 25 arranques y quedarse con el mejor evita caer en un óptimo local malo.
- set.seed() garantiza reproducibilidad. \$cluster guarda la etiqueta de grupo de cada observación.

fviz_cluster()

Visualiza los grupos de k-means proyectándolos sobre el plano PC1-PC2.

Uso en el ejercicio

```
fviz_cluster(km_vinos, data = vinos_norm,  
             geom = "point",  
             ellipse.type = "convex",  
             palette = c("#1e4c7d", "#2d7a7a",  
                        "#c97b2a"))
```

Qué hace

- factoextra usa internamente el PCA para mostrar grupos de muchas dimensiones en un plano de 2D.
- ellipse.type = "convex" dibuja la envolvente de cada grupo, útil para ver solapamientos.
- geom = "point" muestra solo los puntos (sin etiquetas), ideal con cientos de observaciones.

group_by() + summarise(across())

Perfila los grupos: calcula la media de cada variable ORIGINAL por cluster para interpretarlos.

Uso en el ejercicio

```
vinos_perfilados <- vinos %>%  
  mutate(cluster = factor(km_vinos$cluster))  
  
perfil_clusters <- vinos_perfilados %>%  
  group_by(cluster) %>%  
  summarise(across(everything(), mean),  
            n = n())
```

Qué hace

- mutate() pega la etiqueta de grupo a los datos en sus unidades originales (no las normalizadas), para leer cifras reales.
- group_by(cluster) + summarise() resume cada grupo; across(everything(), mean) aplica la media a TODAS las columnas de una vez.
- Comparando las medias se nombra cada grupo: p. ej. 'vinos secos y alcohólicos' vs. 'dulces con mucho azufre'.

dist() + hclust()

Clustering jerárquico: construye un árbol fusionando observaciones por cercanía, sin fijar k de antemano.

Uso en el ejercicio

```
dist_vinos <- dist(vinos_norm,  
  method = "euclidean")  
  
hc_vinos <- hclust(dist_vinos,  
  method = "ward.D2")
```

Qué hace

- dist() arma la matriz de distancias euclidianas entre todas las muestras normalizadas; es el insumo del jerárquico.
- hclust() construye el dendrograma fusionando primero lo más cercano.
- method = "ward.D2" tiende a formar grupos compactos y de tamaño parecido, lo que lo hace comparable con k-means.

fviz_dend() + cutree()

Dibuja el dendrograma y lo 'corta' a la altura que deja el número de grupos deseado.

Uso en el ejercicio

```
fviz_dend(hc_vinos, k = k_elegido,  
          show_labels = FALSE, rect = TRUE)  
  
grupos_hc <- cutree(hc_vinos, k = k_elegido)  
table(kmeans = km_vinos$cluster,  
      jerárquico = grupos_hc)
```

Qué hace

- `fviz_dend()` muestra el árbol; `k` colorea las ramas según el número de grupos y `rect = TRUE` los enmarca.
- `cutree(hc, k)` corta el árbol en `k` grupos y devuelve la etiqueta de cada observación.
- `table()` cruza k-means vs. jerárquico: si cada grupo de un método cae en UNO del otro, ambos descubren la misma estructura.

Gracias

¿Preguntas?